

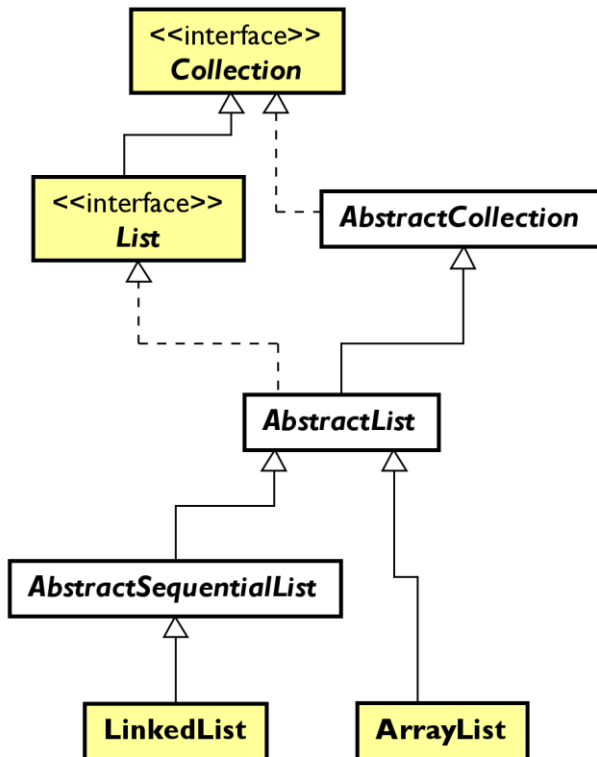
Biblioteca de estrutura de dados

Introdução

- Uma “Estruturas de dados” é uma forma de armazenar e organizar dados na memória do computador.
- “Escolher a melhor estrutura de dados e algoritmos para uma tarefa particular é uma das chaves para o desenvolvimento de software de alta performance” (LIANG, 2019)
- As classes de manipulação de estrutura de dados estão agrupadas em dois grandes grupos: **Coleções** e **Mapas**

Coleções

Principais coleções



ArrayList implementa uma lista estática
LinkedList implementa uma lista dinâmica

Diferenças

Operação	ArrayList	LinkedList
get(index)	Extremamente rápido	Lento
add(E)	Rápido na maior parte das vezes	Extremamente rápido
add(index, E)	Lento	Lento
remove(index, E)	Lento	Lento

Interface Collection

E

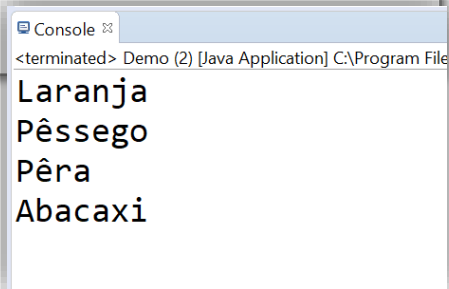
<<interface>> <i>Collection</i>
+ size() : int + isEmpty() : boolean + contains(o : Object) : boolean + iterator() : Iterator<E> + toArray() : Object[] + toArray(a : T[]) : T[] + add(e : E) : boolean + remove(o : Object) : boolean + containsAll(c : Collection<?>) : boolean + addAll(c : Collection<E>) : boolean + removeAll(c : Collection<?>) : boolean + retainAll(c : Collection<?>) : boolean + clear() : void + equals(o : Object) : boolean + hashCode() : int

Método	Descrição
size()	Retorna a quantidade de elementos armazenados
isEmpty()	Retorna true se a estrutura estiver vazia
contains(Object)	Retorna true se o objeto está armazenado na estrutura
iterator()	Retorna um objeto que permite percorrer a estrutura
toArray()	Devolve um vetor com os dados da estrutura
add(E)	Adiciona o objeto na estrutura
remove(Object)	Remove o objeto da estrutura
containsAll(Collection)	Retorna true se todos os elementos pertencerem à coleção
addAll(Collection)	Adiciona todos os elementos na coleção
removeAll	Remove todos os elementos que pertencem à coleção
retainAll(Collection)	Mantém apenas os elementos que estão na coleção informada como parâmetro
clear()	Remove os elementos da coleção

Exemplo

- O **ArrayList** implementa indiretamente **Collection**:

```
Collection<String> frutas = new ArrayList<>();  
frutas.add("Laranja");  
frutas.add("Morango");  
frutas.add("Pêssego");  
frutas.add("Pêra");  
frutas.add("Abacaxi");  
frutas.remove("Morango");  
  
for (String fruta: frutas) {  
    System.out.println(fruta);  
}  
  
frutas.clear();
```



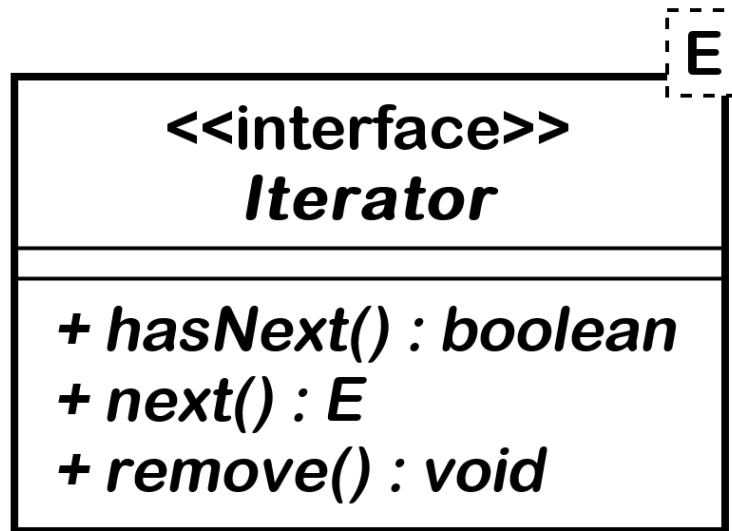
Console

<terminated> Demo (2) [Java Application] C:\Program File

Laranja
Pêssego
Pêra
Abacaxi

O método iterator()

- A interface **Collection** prevê que todas as classes concretas que a implemente, retorne um *iterador*, através do método **iterator()**.



Um *iterador* é um objeto que possibilita percorrer uma estrutura de dados.

Método	Descrição
hasNext()	Retorna true se tem objetos na coleção ainda não lidos
next()	Retorna um objeto da coleção
remove()	Remove o último objeto lido pelo iterador (através de next()), da coleção

Utilidade do iterador

Remover as frutas que começam com “P”:

```
Collection<String> frutas = new ArrayList<>();  
frutas.add("Laranja");  
frutas.add("Morango");  
frutas.add("Pêssego");  
frutas.add("Pêra");  
frutas.add("Abacaxi");  
  
for (String fruta : frutas) {  
    if (fruta.startsWith("P")) {  
        frutas.remove(fruta);  
    }  
}
```

Não é seguro, pois lança a exceção:

```
Exception in thread "main" java.util.ConcurrentModificationException  
at java.base/java.util.ArrayList$Itr.checkForComodification(ArrayList.java:1042)  
at java.base/java.util.ArrayList$Itr.next(ArrayList.java:996)  
at teste.Demo.main(Demo.java:18)
```


Utilidade do iterador

- Usando o iterador:

```
Collection<String> frutas = new ArrayList<>();
frutas.add("Laranja");
frutas.add("Morango");
frutas.add("Pêssego");
frutas.add("Pêra");
frutas.add("Abacaxi");

Iterator<String> iterador = frutas.iterator();
while (iterador.hasNext()) {
    String fruta = iterador.next();
    if (fruta.startsWith("P"))
        iterador.remove();
}

for (String fruta : frutas) {
    System.out.println(fruta);
}
```

```
Console
<terminated> Demo (2) [Java Application]
Laranja
Morango
Abacaxi
```

Interface List

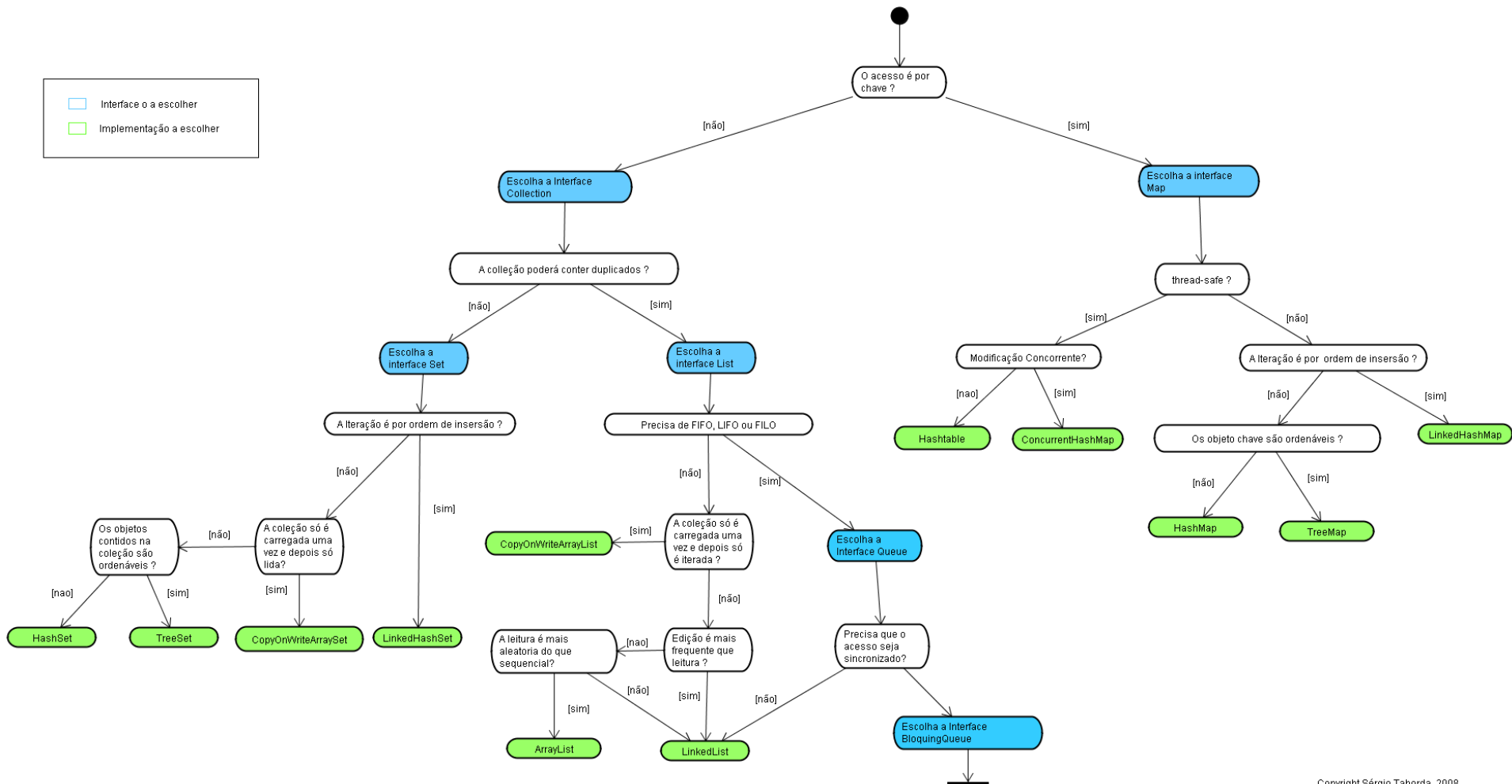
<<interface>> List
+ get(index : int) : E + set(index : int, element : E) : E + add(index : int, element : E) : void + indexOf(o : Object) : int + lastIndexOf(o : Object) : int + listIterator() : ListIterator<E> + subList(fromIndex : int, toIndex : int) : List<E>

Método	Descrição
get(int)	Retorna o objeto que ocupa a posição informada
set()	Altera o objeto armazenado na posição indicada
add()	Acrescenta um objeto na posição indicada
indexOf()	Procura por um objeto
lastIndexOf()	Retorna a posição da última ocorrência do objeto
listIterator()	Devolve um iterador que suporta navegação bidirecional
subList()	Retorna uma sub-lista

<<interface>> ListIterator
+ hasNext() : boolean + next() : E + hasPrevious() : boolean + previous() : E + nextIndex() : int + previousIndex() : int + remove() : void

Interface que prevê a navegação bidirecional

Escolha de Collection



Copyright Sérgio Taborda, 2008